# Software Design Specification

**for**

# <Student Grade & GPA Calculator>

**Version 1.0 approved**

**<date created: 4/21/2026>**

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

# TABLE OF CONTENTS


***1.0 INTRODUCTION*** ..... ***3***

***1.1 Purpose*** ..... ***3***

***1.2 Scope*** ..... ***3***

***1.3 Overview*** ..... ***3***

***1.4 Reference Material*** ..... ***3***

***1.5 Definitions and Acronyms*** ..... ***3***

***2.0 SYSTEM OVERVIEW*** ..... ***4***

***3.0 SYSTEM ARCHITECTURE*** ..... ***4***

***3.1 Architectural Design*** ..... ***4***

***3.2 Decomposition Description*** ..... ***6***

***3.3 Design Rationale*** ..... ***12***

***4.0 DATA DESIGN*** ..... ***12***

***4.1 Data Description*** ..... ***12***

***4.2 Data Dictionary*** ..... ***12***

***5.0 COMPONENT DESIGN*** ..... ***15***

# 1.0 INTRODUCTION

## 1.1 Purpose

This Software Design Description Document (SWDD) provides a detailed description of the architecture and internal design of the **Student Grade & GPA Calculator** system. It explains how the system is structured, how its components interact, and how the required functionalities are implemented.

This document is intended for:

- Developers responsible for building and maintaining the system
- Testers who will verify system behavior and correctness
- Instructors and teaching assistants evaluating the project
- Team members who need a shared and consistent understanding of the system design

The document focuses on the system's internal organization, including its modules, data structures, and interactions between different components such as controllers, screens, and data entities.

## 1.2 Scope

The Student Grade & GPA Calculator is a web-based application that allows students to calculate their GPA based on course-related data such as grades, marks, and credit hours.

The system provides a simple and interactive interface where users can enter course information and obtain GPA results. It focuses on accurate calculation and ease of use without relying on external systems or permanent data storage.

## 1.3 Overview

- Section 1 introduces the document, including its purpose, scope, and references
- Section 2 presents a high-level overview of the system
- Section 3 describes the system architecture and design decisions
- Section 4 focuses on data design and system entities
- Section 5 explains component design, including classes and interactions

## 1.4 Reference Material

- IEEE 830 Software Requirements Specification standard
- JavaScript, HTML, and CSS documentation
- Object-Oriented Analysis and Design principles
- UML guidelines for class and sequence diagrams
- Course materials for Software Process & Quality Management

### 1.5 Definitions and Acronyms

- SWDD: Software Design Description Document
- SRS: Software Requirements Specification
- SDS: Software Design Specification
- UML: Unified Modeling Language
- Student: The primary user who interacts with the system to perform GPA calculations
- Course: A unit of study defined by a name, credit hours, and evaluation (mark or grade)
- Grade: A letter-based representation of performance (e.g., A, B, C)
- Mark: A numeric value representing student performance (e.g., 85, 90)
- GPA (Grade Point Average): A weighted average calculated based on course grades and credit hours
- Credit Hours: A value representing the weight of a course in GPA calculation
- Calculation Type: The selected method (marks or grades) used for GPA computation

## 2.0 SYSTEM OVERVIEW

The Student Grade & GPA Calculator is a web-based application designed to simplify the process of calculating academic grades and Grade Point Average (GPA). The system operates through a graphical user interface and allows students to perform all calculations directly within the browser.

The system is intended for a single primary user type, which is the **student**. The student interacts with the system by entering course information and selecting the desired calculation method.

The system provides the following core functionalities:

- Converting numeric marks into corresponding letter grades based on a predefined grading scale
- Calculating GPA using letter grades and credit hours
- Calculating GPA using numeric marks by converting them into grades first
- Managing course data, including adding and deleting courses
- Displaying calculated GPA results along with relevant course details

The system guides the user through a sequence of screens, starting from the main menu, followed by course input, processing, and result display. All interactions are handled dynamically, and the system provides immediate feedback such as validation messages and calculated results.

Unlike large-scale systems, this application does not support multiple user roles, external integrations, or persistent data storage. All data is entered by the user during runtime and is cleared after the session ends.

## 3.0 SYSTEM ARCHITECTURE

### 3.1 Architectural Design

The system follows a **modular, layered architecture** that separates the user interface, control logic, and data handling components. This separation improves maintainability, readability, and scalability.

The system is divided into the following major subsystems:

## 1. User Interface (UI) Subsystem

This subsystem handles all user interactions through different screens. It is responsible for displaying information and collecting user input.

Main components include:

- Main Menu Screen (navigation and option selection)
- Course Input Screen (entering course details)
- Course List Screen (displaying and managing courses)
- Grade Conversion Screen (converting marks to grades)
- GPA Calculation Screen (handling GPA input and display)
- Result Screen (showing final GPA and results)

This subsystem ensures a smooth and user-friendly interaction flow between the user and the system.

---

## 2. Control Logic Subsystem

This subsystem manages the system's behavior and coordinates interactions between the user interface and data components. It contains the core logic that processes user actions and controls the application flow.

Main components include:

- Calculation Controller (manages calculation type and system flow)
- Course Controller (handles course creation, deletion, and updates)
- Grade Controller (validates marks and converts them into grades)
- GPA Controller (validates inputs and calculates GPA values)

This subsystem acts as the central processing unit of the application.

---

## 3. Data Management Subsystem

This subsystem is responsible for storing and managing application data during runtime. It represents the core data structures used by the system.

Main components include:

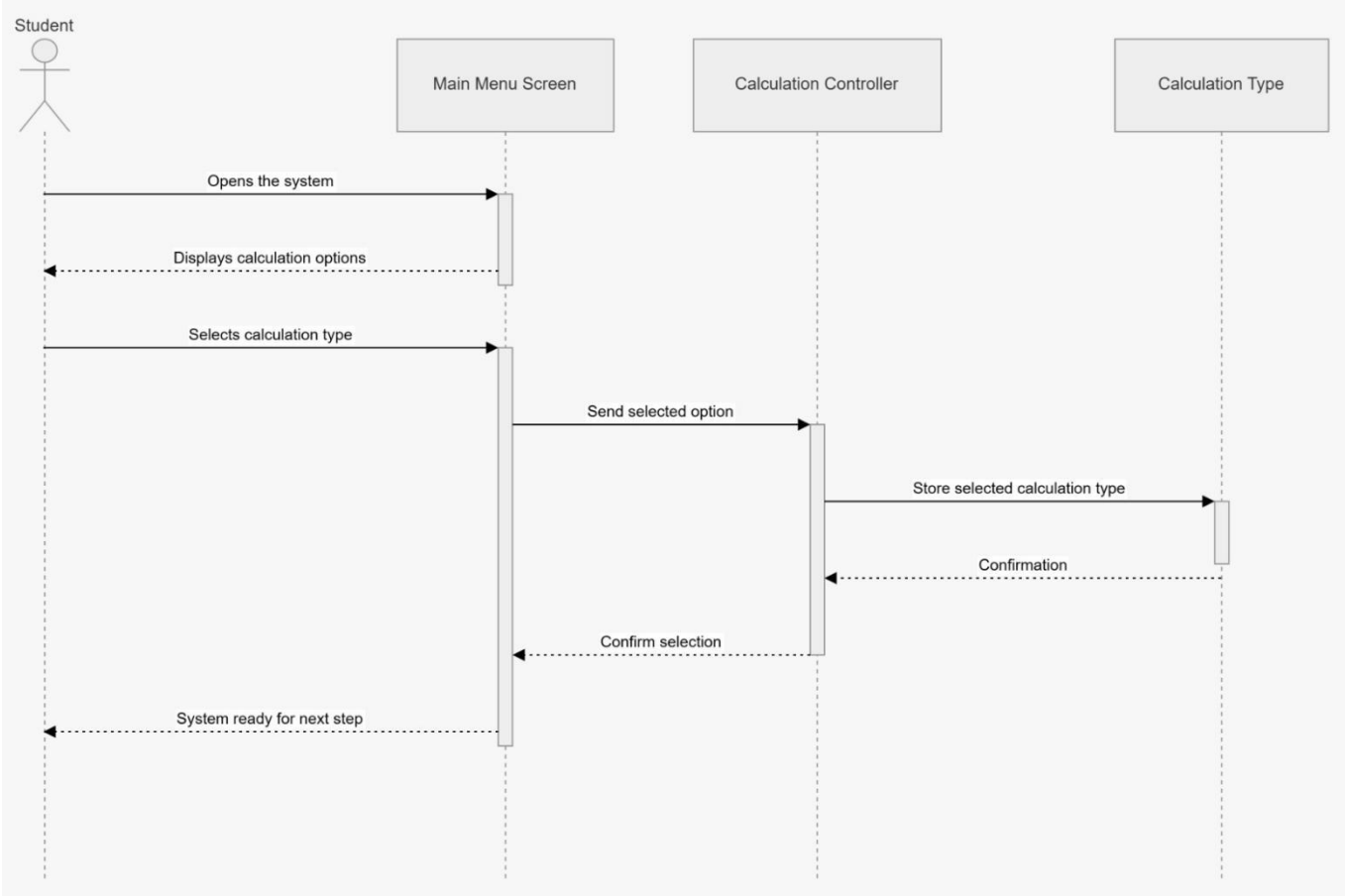
- Course (stores course name, credit hours, mark, and grade)
- Course List (manages a collection of courses)
- Grade Scale (defines grade ranges and corresponding points)
- Course Records (stores course data during the session)
- Calculation Session (manages the current calculation state)

All data is stored temporarily in memory and is cleared when the session ends

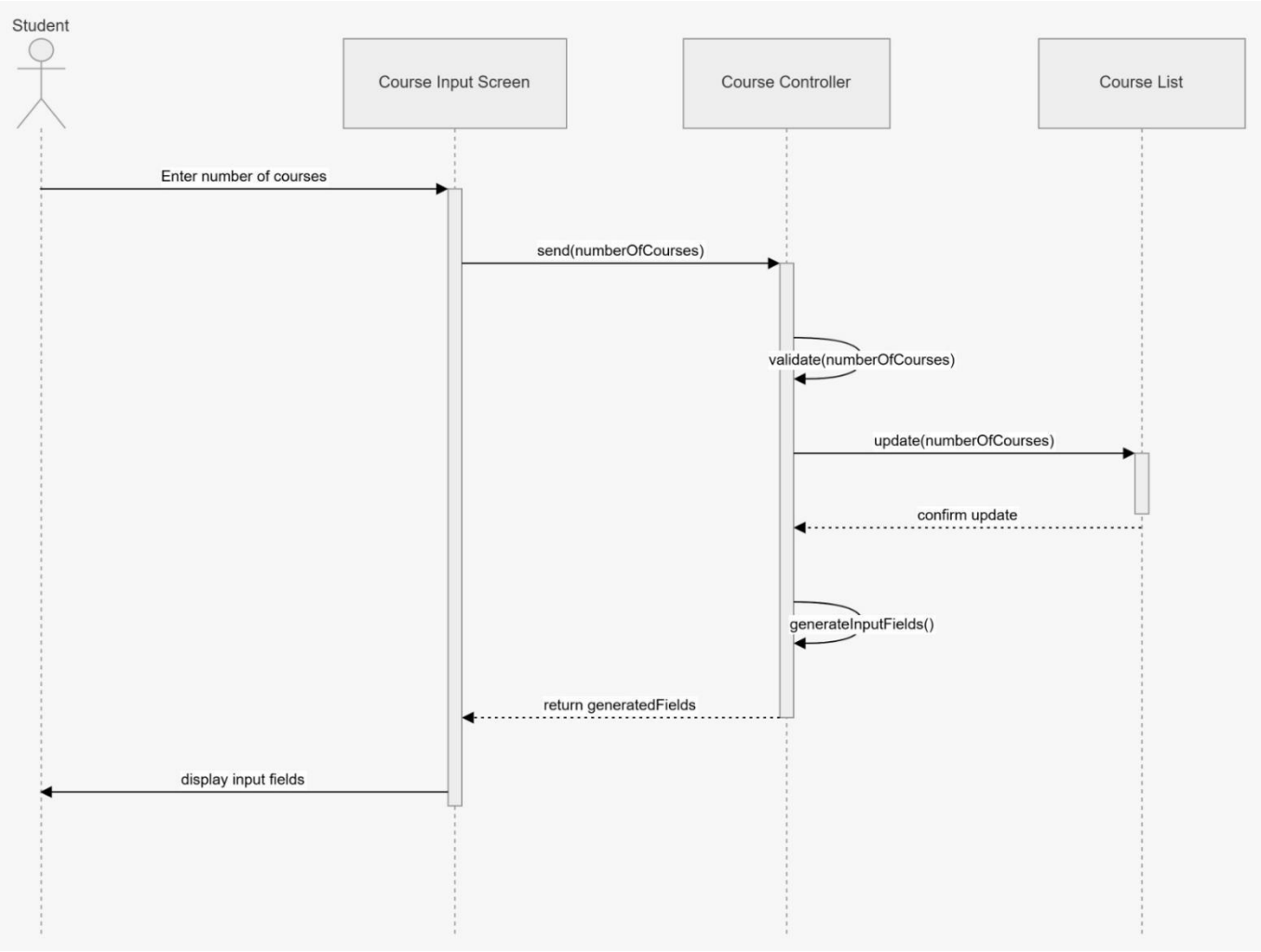
- 

## 3.2 Decomposition Description

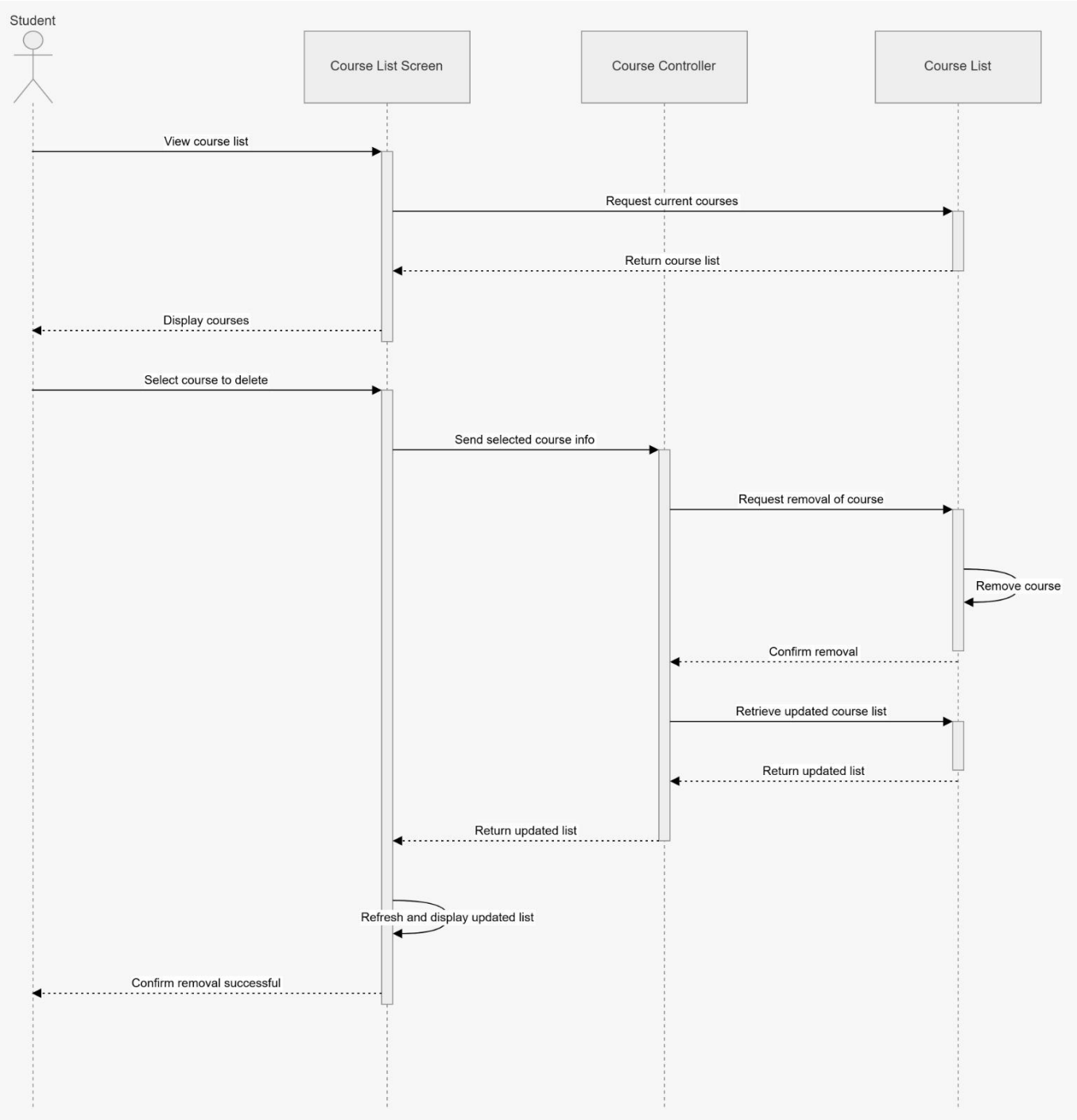
# UC-01



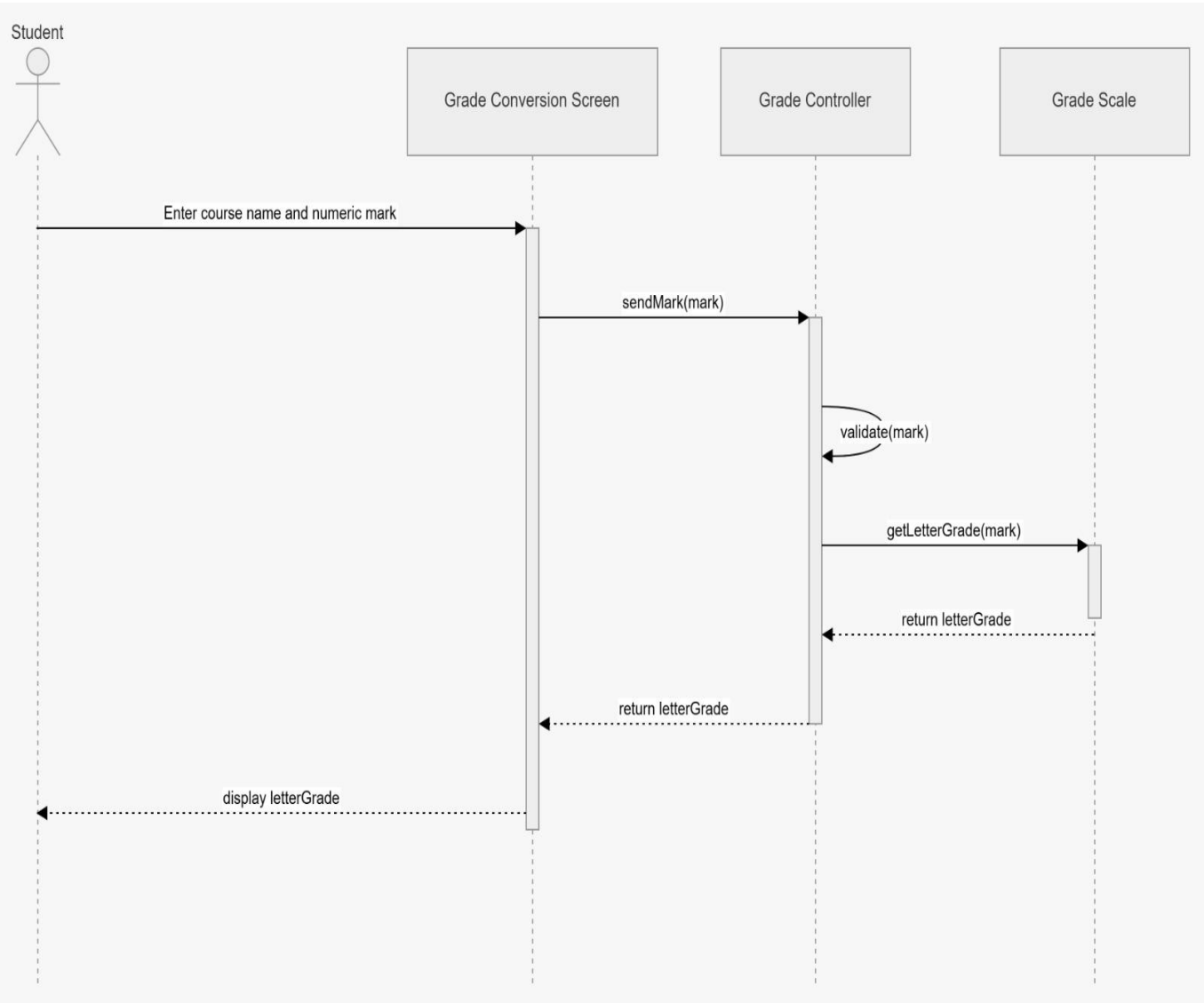
# UC-02


Student
Course Input Screen
Course Controller
Course List
Enter number of courses
send(numberOfCourses)
validate(numberOfCourses)
update(numberOfCourses)
confirm update
generateInputFields()
return generatedFields
display input fields

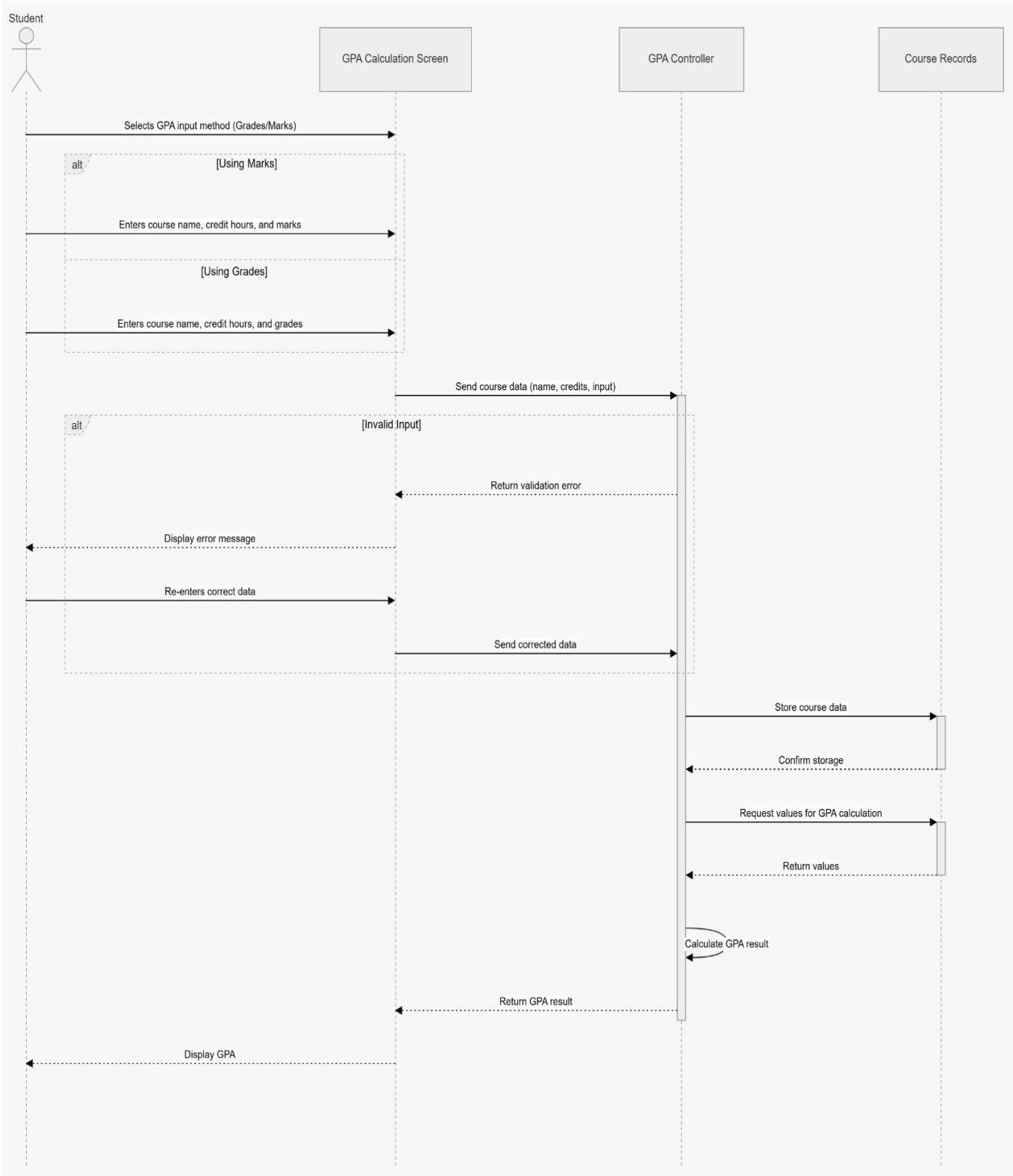
# UC-03


Student
Course List Screen
Course Controller
Course List
View course list
Request current courses
Return course list
Display courses
Select course to delete
Send selected course info
Request removal of course
Remove course
Confirm removal
Retrieve updated course list
Return updated list
Return updated list
Refresh and display updated list
Confirm removal successful

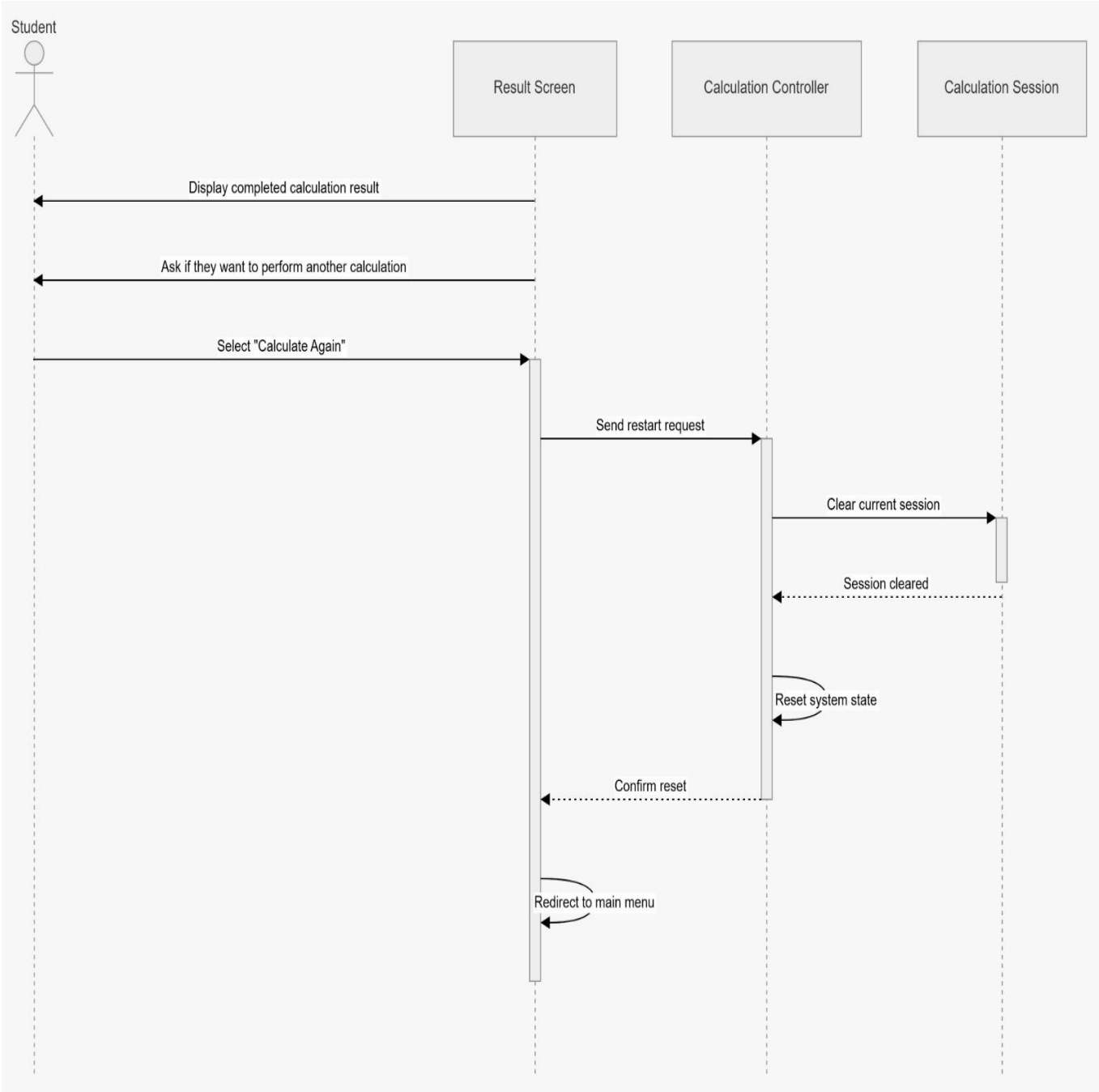
# UC-04


Student
Grade Conversion Screen
Grade Controller
Grade Scale
Enter course name and numeric mark
sendMark(mark)
validate(mark)
getLetterGrade(mark)
return letterGrade
return letterGrade
display letterGrade

# UC-05


Student
GPA Calculation Screen
GPA Controller
Course Records
Selects GPA input method (Grades/Marks)
alt
[Using Marks]
Enters course name, credit hours, and marks
[Using Grades]
Enters course name, credit hours, and grades
Send course data (name, credits, input)
alt
[Invalid Input]
Return validation error
Display error message
Re-enters correct data
Send corrected data
Store course data
Confirm storage
Request values for GPA calculation
Return values
Calculate GPA result
Return GPA result
Display GPA

# UC-06


Student
Result Screen
Calculation Controller
Calculation Session
Display completed calculation result
Ask if they want to perform another calculation
Select "Calculate Again"
Send restart request
Clear current session
Session cleared
Reset system state
Confirm reset
Redirect to main menu

### 3.3 Design Rationale

A modular architecture was selected to organize the system into clear layers, including user interface components, control logic, and data models. This structure supports separation of concerns, making the system easier to understand, develop, and maintain.

The use of controllers allows the system to manage user interactions and application flow efficiently, while keeping the user interface independent from the core logic. Data-related classes are separated to ensure that information is handled in a structured and consistent way.

Object-oriented design principles such as encapsulation and modularity are applied to reduce code duplication and improve reusability. This approach also simplifies testing and future modifications, as changes in one part of the system have minimal impact on other components.

## 4.0 DATA DESIGN

### 4.1 Data Description

The data in the Student Grade & GPA Calculator is represented using object-oriented structures. Each major element of the system is modeled as a class that stores and manages specific types of information.

The main system entities include:

- Student (represents the user interacting with the system)
- Course (stores course name, credit hours, mark, and grade)
- Course List (manages a collection of courses entered by the user)
- Grade Scale (defines the mapping between marks, letter grades, and grade points)
- Course Records (stores course data during the current calculation session)
- Calculation Session (represents the current state of the system during runtime)
- Calculation Type (stores the selected calculation method: marks or grades)

All data in the system is handled temporarily during execution and is not stored permanently.

The system does not use any external database. Instead, data is managed in-memory using data structures within the application. All entered information is cleared once the session ends or when the user starts a new calculation.

### 4.2 Data Dictionary

**Actors**

**Student** — Attributes: none
Methods: selectCalculationType(), enterCourseData(), deleteCourse(), requestNewCalculation()

---

**User Interface Components**

**MainMenuScreen** — Attributes: selectedOption
Methods: displayMenu(), getSelectedOption(), showNextScreen()

**CourseInputScreen** — Attributes: numberOfCourses, courseFields
Methods: promptCourseCount(), generateCourseFields(), displayCourseFields()

**CourseListScreen** — Attributes: courseList
Methods: displayCourses(), selectCourseToDelete(), refreshCourseList()

**GradeConversionScreen** — Attributes: courseName, mark, letterGrade
Methods: inputMark(), displayGrade()

**GPACalculationScreen** — Attributes: selectedInputMethod, enteredCourses, calculatedGPA
Methods: chooseInputMethod(), collectCourseData(), displayGPA(), displayError()

**ResultScreen** — Attributes: result
Methods: showResult(), askForNewCalculation(), redirectToMainMenu()

---

**Control Components**

**CalculationController** — Attributes: selectedCalculationType
Methods: processCalculationType(), resetCalculation(), controlFlow()

**CourseController** — Attributes: totalCourses
Methods: validateCourseCount(), createCourseFields(), deleteCourse(), updateCourseList()

**GradeController** — Attributes: gradingScale
Methods: validateMark(), convertMarkToGrade()

**GPAController** — Attributes: courseRecords, gpaValue
Methods: validateInputs(), calculateGPA(), convertMarksToPoints()

---

**Data / Model Components**

**CalculationType** — Attributes: typeName
Methods: storeSelectedType()

**Course** — Attributes: courseName, creditHours, mark, grade
Methods: setCourseData(), getCourseData()

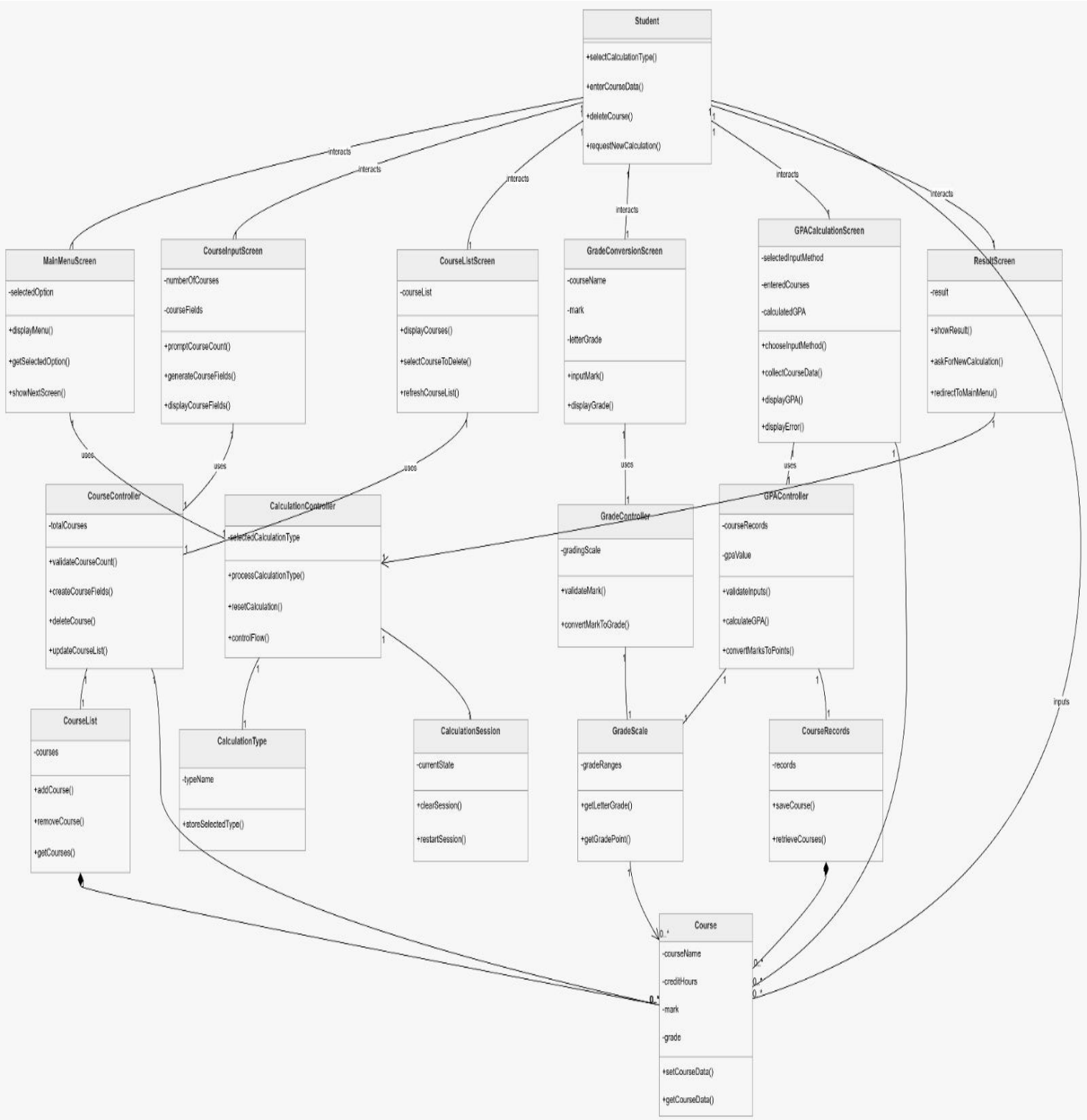
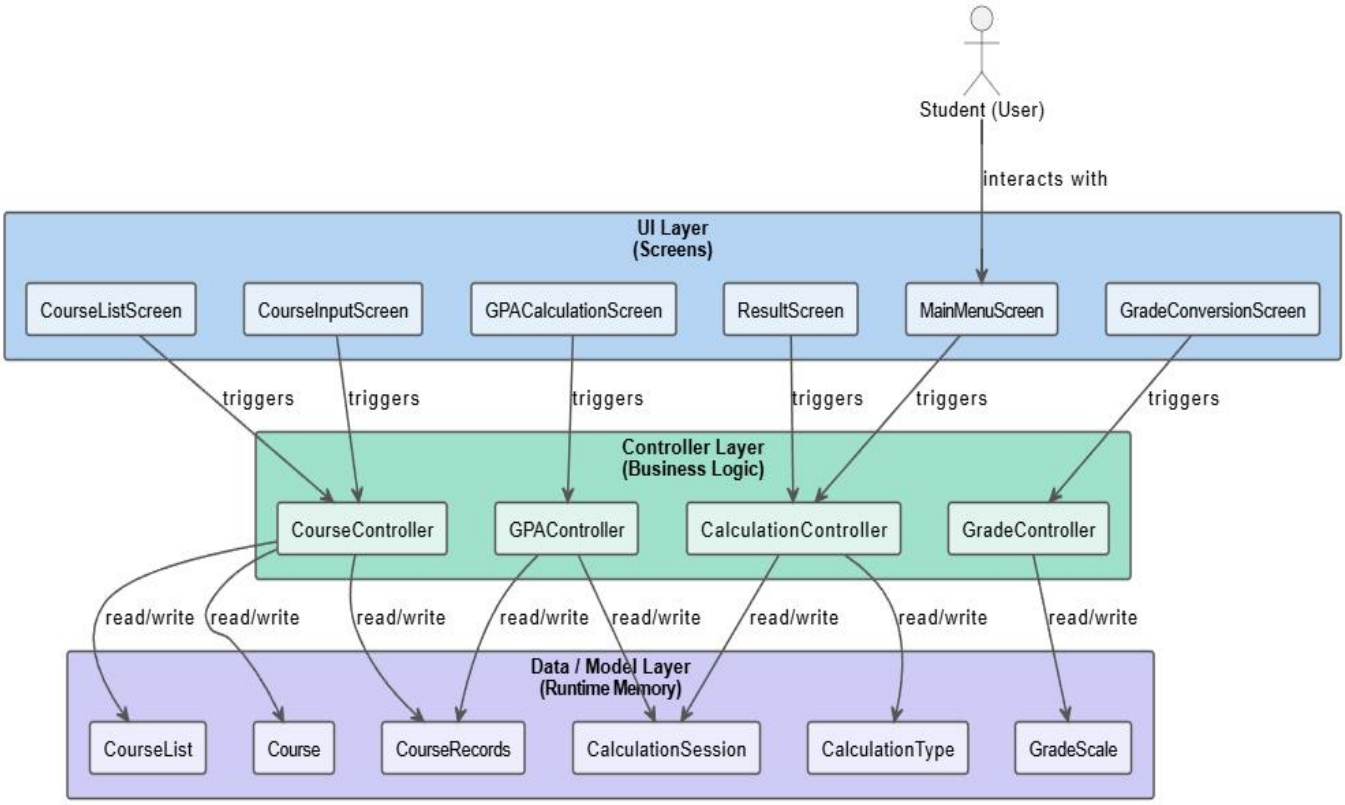
**CourseList** — Attributes: courses
Methods: addCourse(), removeCourse(), getCourses()

**GradeScale** — Attributes: gradeRanges
Methods: getLetterGrade(), getGradePoint()

**CourseRecords** — Attributes: records
Methods: saveCourse(), retrieveCourses()

**CalculationSession** — Attributes: currentState
Methods: clearSession(), restartSession()

# 5.0 COMPONENT DESIGN



# 6.0 Architecture diagram